

Actividad 1 – Creación de la Arquitectura y Desarrollo del Proceso de Extracción

Sistema de Análisis de Opiniones de Clientes

Componente: SistemaOpiniones.Etl — Worker Service en .NET 8 | Alcance: fase de extracción (E) del proceso ETL

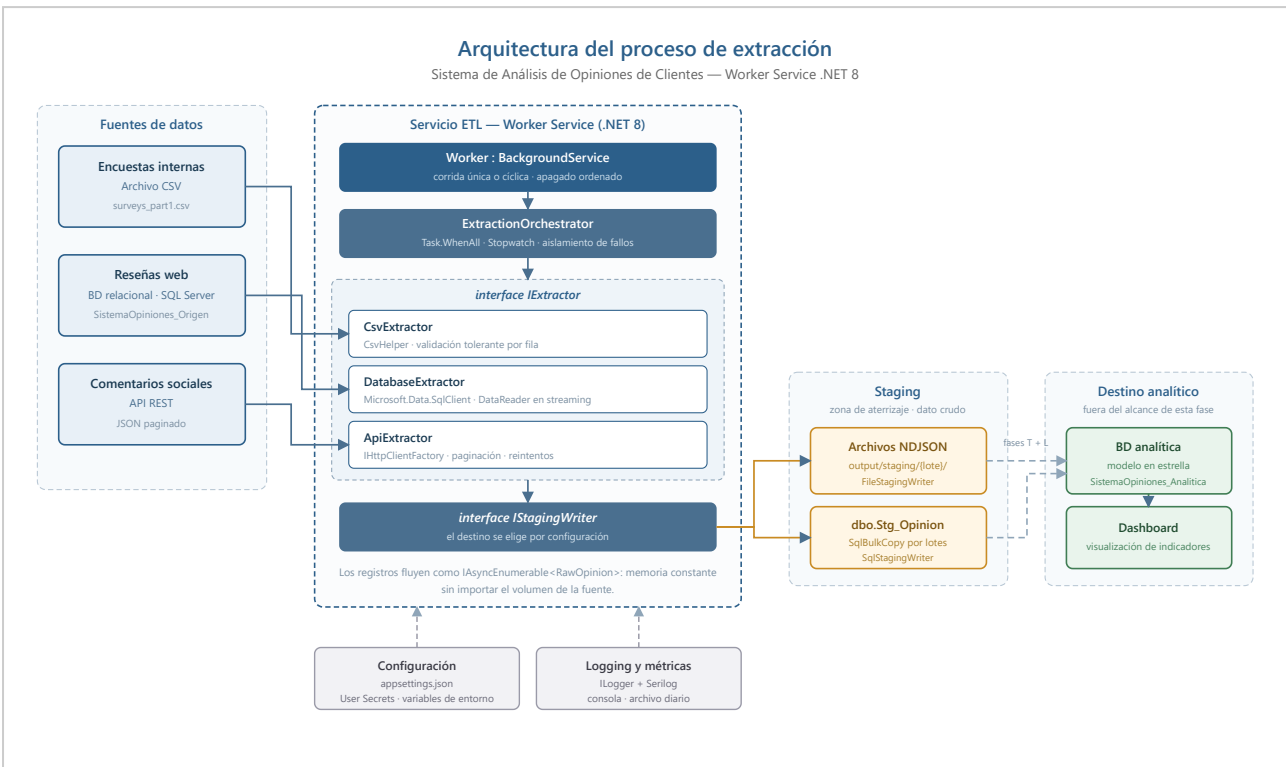
1. Objetivo y alcance

Diseñar la arquitectura base del proceso ETL y desarrollar la **fase de extracción** correspondiente al proyecto, garantizando que la solución cumpla los atributos de calidad definidos en los requisitos no funcionales: rendimiento, escalabilidad, seguridad y mantenibilidad.

El sistema consolida las opiniones de clientes de una empresa de comercio electrónico recogidas por tres canales distintos: **encuestas internas** de satisfacción (archivos CSV), **reseñas publicadas en el sitio web** (base de datos relacional) y **comentarios de usuarios en redes sociales** (API REST).

Esta entrega cubre exclusivamente la extracción: leer las tres fuentes y aterrizar el dato crudo en una zona de *staging*. La transformación y la carga hacia el modelo analítico en estrella corresponden a las actividades siguientes.

2. Diagrama de arquitectura

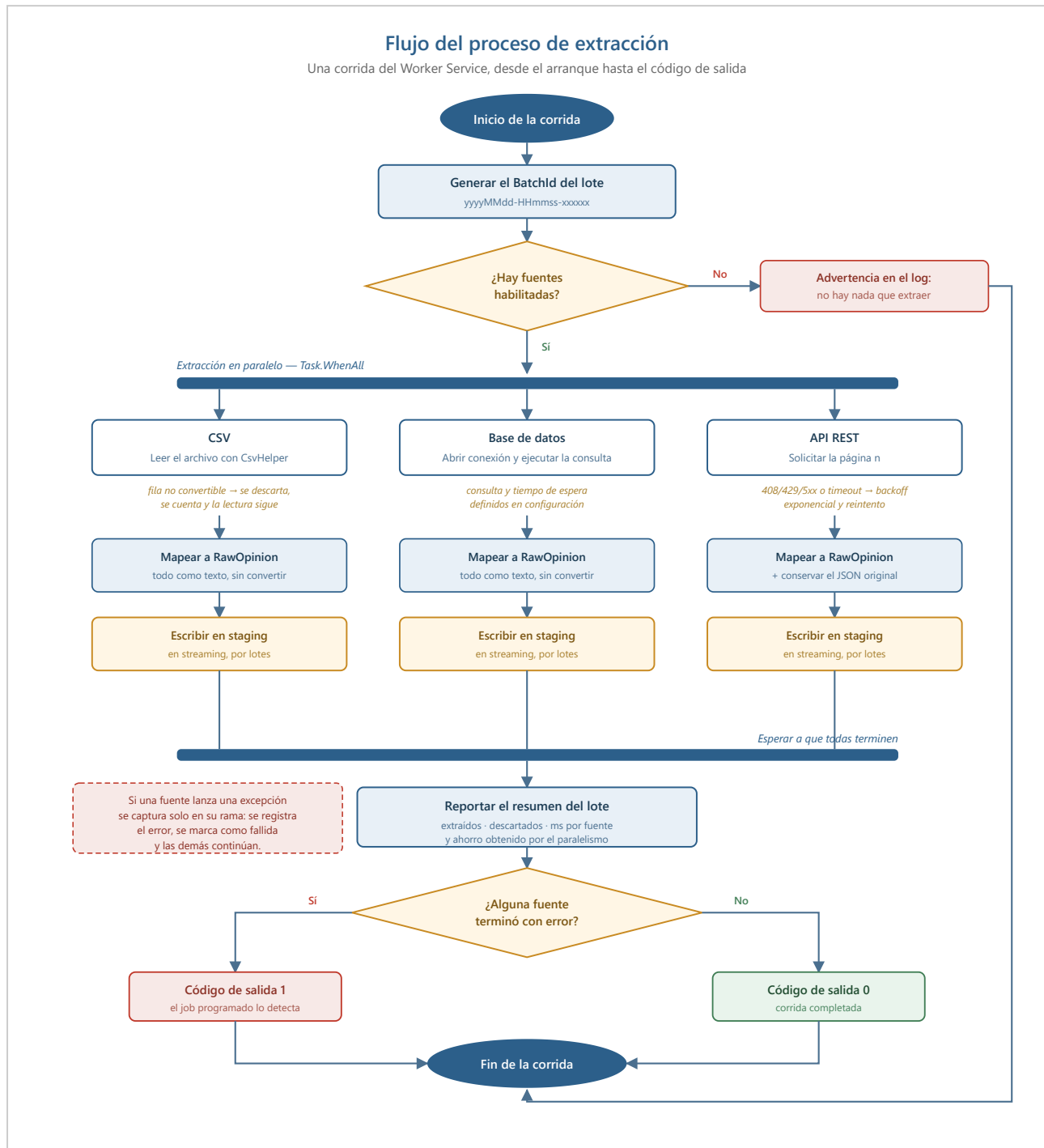


Arquitectura orientada a servicios del proceso de extracción. Las flechas punteadas hacia la base analítica marcan el límite de esta actividad: la extracción termina en staging.

2.1 Componentes

Componente	Tecnología	Responsabilidad
Worker	BackgroundService	Hospeda el proceso; corrida única o cíclica, con apagado ordenado.
ExtractionOrchestrator	C# · Task.WhenAll · Stopwatch	Identifica el lote, lanza las fuentes en paralelo, mide y reporta.
CsvExtractor	C# · CsvHelper	Lee y valida el archivo de encuestas internas.
DatabaseExtractor	C# · Microsoft.Data.SqlClient	Ejecuta la consulta de extracción sobre la BD de origen.
ApiExtractor	C# · IHttpClientFactory	Consume la API REST paginada, con reintentos ante fallos transitorios.
FileStagingWriter	System.Text.Json	Aterrizo lo extraído en archivos NDJSON.
SqlStagingWriter	ADO.NET · SqlBulkCopy	Aterrizo lo extraído en la tabla dbo.Stg_Opinion .
Servicio de logging	ILogger + Serilog	Traza de eventos, errores y métricas de rendimiento.

3. Diagrama de flujo del proceso



Recorrido completo de una corrida del Worker Service, desde el arranque hasta el código de salida.

4. Justificación de las decisiones técnicas

4.1 Worker Service en lugar de aplicación de consola

La extracción es un proceso recurrente y desatendido. `BackgroundService` aporta el ciclo de vida completo (arranque, apagado ordenado mediante `CancellationToken`), inyección de dependencias y configuración por capas sin escribir nada de eso a mano. Con `AddWindowsService()` el mismo binario se instala como servicio de Windows sin cambiar código, y con `RunOnceAndExit` sirve para ejecución programada. Una sola base de código cubre los dos modos de operación.

4.2 `IExtractor` : una abstracción por fuente

Las tres fuentes no se parecen en nada —un archivo, un `SqlDataReader` y una API paginada— pero desde el punto de vista del proceso hacen lo mismo: producir registros. Esa es exactamente la definición de la interfaz:

```
public interface IExtractor
{
    string SourceName { get; }
    string SourceType { get; }
    bool Enabled { get; }
    IAsyncEnumerable<RawOpinion> ExtractAsync(ExtractionContext context, CancellationToken
cancellationToken);
}
```

El orquestador recibe `IEnumerable<IExtractor>` y no conoce ninguna implementación concreta. Es el **principio de inversión de dependencias** aplicado literalmente: la política (orquestar, medir, reportar) no depende del detalle (cómo se lee un CSV). También cumple el **principio abierto/cerrado**: sumar una cuarta fuente no modifica ninguna clase existente, solo agrega una y una línea de registro en el contenedor.

4.3 `IAsyncEnumerable` en vez de devolver listas

Un extractor que devolviera `Task<List<RawOpinion>>` tendría que materializar la fuente completa en memoria antes de escribir el primer registro. Devolviendo `IAsyncEnumerable<RawOpinion>` los registros fluyen de la fuente al destino de a uno: el consumo de memoria no depende del tamaño del archivo ni de cuántas filas devuelva la consulta, y la escritura comienza mientras la lectura sigue en curso.

4.4 Staging crudo, sin conversión de tipos

`RawOpinion` guarda todos los campos como texto y la tabla `Stg_Opinion` los declara `NVARCHAR`. Es deliberado: si la extracción intentara convertir tipos, un cambio de formato de fecha en el origen haría **perder** el registro en lugar de registrarlo. Al aterrizar crudo, el dato siempre queda persistido y la fase de transformación decide qué hacer con él, con la evidencia original disponible.

Por la misma razón el `ApiExtractor` conserva el JSON completo en `RawPayload`: si el mapeo resulta incompleto, el dato está en staging y no hay que volver a llamar la API.

4.5 `IStagingWriter` con dos implementaciones

El destino es una decisión de configuración, no de código. `FileStagingWriter` (NDJSON) permite ejecutar y evidenciar la extracción sin depender de que haya un servidor disponible; `SqlStagingWriter` es el destino de producción, con `SqlBulkCopy`. Los extractores no saben cuál de los dos está activo.

Se eligió NDJSON —un objeto JSON por línea— y no un arreglo JSON único porque permite escribir en streaming y volver a leer el archivo por partes.

4.6 Mapeo de la API dirigido por configuración

`ApiExtractor` no tiene una clase que refleje el contrato de la API. Lee el JSON con `JsonDocument` y resuelve cada campo con el diccionario `FieldMap` de `appsettings.json`:

```
"FieldMap": {  
  "ExternalId": "id",  
  "ClienteRef": "email",  
  "ProductoRef": "postId",  
  "Comentario": "body"  
}
```

Un cambio en el contrato de la API, o el cambio de proveedor, se absorbe editando configuración. Es la parte del sistema con mayor probabilidad de cambiar y la que menos código requiere tocar.

4.7 `IHttpClientFactory` y no `new HttpClient()`

Instanciar `HttpClient` por llamada agota los sockets del sistema; reusarlo como campo estático deja de ver los cambios de DNS. `IHttpClientFactory` resuelve ambos problemas: agrupa y recicla los handlers en un intervalo configurado.

4.8 Reintentos solo ante fallos transitorios

El `ApiExtractor` reintenta con retroceso exponencial (1 s, 2 s, 4 s) ante los códigos 408, 429 y 5xx, y ante timeouts. Un 401 o un 404 **no** se reintentan: no se arreglan esperando y reintentarlos solo retrasa el diagnóstico.

5. Cumplimiento de los atributos de calidad

5.1 Rendimiento

Mecanismo	Dónde	Efecto
Extracción en paralelo	<code>ExtractionOrchestrator</code> → <code>Task.WhenAll</code>	La corrida dura lo que la fuente más lenta, no la suma de todas.
E/S asíncrona de punta a punta	<code>ExtractAsync</code> , <code>OpenAsync</code> , <code>ReadAsync</code> , <code>WriteLineAsync</code>	Ningún hilo queda bloqueado esperando disco, red o base de datos.
Streaming con <code>IAsyncEnumerable</code>	los tres extractores	Memoria constante, independiente del volumen.
<code>SqlBulkCopy</code> por lotes	<code>SqlStagingWriter</code>	Carga masiva por protocolo TDS en vez de un <code>INSERT</code> por fila.
Buffer de 64 KB y vaciado por lote	<code>FileStagingWriter</code>	Menos llamadas al sistema de archivos.
Medición con <code>Stopwatch</code>	por fuente y total	Métrica objetiva: registros/segundo y ahorro por paralelismo.
Límite de concurrencia	<code>SemaphoreSlim</code> configurable	Evita saturar la red o los servidores de origen al crecer en fuentes.

5.2 Escalabilidad

- **Nuevas fuentes:** implementar `IExtractor` y registrarla. Ni el orquestador ni el Worker ni los escritores de staging cambian.
- **Encender y apagar canales:** la propiedad `Enabled` de cada fuente, en configuración, sin recompilar ni redespargar.
- **Nuevo destino de staging:** implementar `IStagingWriter` (por ejemplo, hacia almacenamiento en la nube) sin tocar los extractores.
- **Volumen:** el streaming y la carga por lotes hacen que crecer en registros no altere el perfil de memoria; solo el tiempo.
- **Extracción incremental:** la consulta de la fuente de base de datos está en configuración, de modo que filtrar por fecha de última carga es un cambio de texto.

5.3 Seguridad

- **Ninguna credencial en el repositorio.** `appsettings.json` contiene *nombres* de cadenas de conexión, nunca valores con credenciales. Los secretos llegan por **User Secrets** en desarrollo y por **variables de entorno** en despliegue, que el host incorpora a la cadena de configuración con precedencia sobre el archivo.

- **Sin concatenación de SQL.** La consulta de extracción se define en configuración y se ejecuta tal cual; el proceso no arma SQL con datos de entrada, de modo que no hay superficie de inyección. La carga usa procedimientos almacenados con parámetros tipados.
- **Credencial de la API fuera del código y fuera del log.** Se inyecta como cabecera al registrar el `HttpClient` y nunca se escribe en la traza.
- **Conexiones cifradas.** Las cadenas de conexión usan `Encrypt=True`.
- **Datos de clientes fuera del control de versiones.** El archivo `.gitignore` excluye los archivos de origen reales y las salidas de staging; el repositorio solo lleva una muestra.

5.4 Mantenibilidad

- **Separación por capas y responsabilidad única:** `Worker` (ciclo de vida) → `ExtractionOrchestrator` (coordinación) → `IExtractor` (acceso a la fuente) → `IStagingWriter` (persistencia). Cada clase tiene un solo motivo para cambiar.
- **Configuración tipada:** `Et1Options` enlaza la sección completa y se inyecta con `IOptions<T>`; no hay lectura de configuración dispersa por el código.
- **Aislamiento de fallos:** cada fuente se extrae dentro de su propio `try/catch`. Si la API está caída, las encuestas y las reseñas se extraen igual y el resumen indica exactamente qué falló y por qué.
- **Trazabilidad:** cada registro lleva `BatchId`, `SourceName` y `ExtractedAtUtc`; siempre se puede reconstruir de dónde salió un dato y en qué corrida.
- **Logging por abstracción:** el código depende de `ILogger`; Serilog está detrás y sus destinos se declaran en configuración. Cambiar de proveedor no toca el código.
- **Reutilización:** el `CsvExtractor` reutiliza `SurveyDto` de la capa de datos, que ya declara el mapeo de las cabeceras con tilde (`Clasificación`, `PuntajeSatisfacción`) mediante atributos de `CsvHelper`.

6. Evidencia del código

6.1 Orquestación paralela y medición

SistemaOpiniones.Etl/Services/ExtractionOrchestrator.cs

```
var stopwatch = Stopwatch.StartNew();

using var gate = _options.MaxDegreeOfParallelism > 0
    ? new SemaphoreSlim(_options.MaxDegreeOfParallelism)
    : null;

var results = await Task.WhenAll(
    enabled.Select(extractor => ExtractSourceAsync(extractor, batchId, startedAtUtc, gate,
        cancellationToken)));

stopwatch.Stop();
LogSummary(batchId, results, stopwatch.ElapsedMilliseconds);
```

Aislamiento de fallos por fuente:

```
catch (Exception ex)
{
    stopwatch.Stop();
    _logger.LogError(ex, "[{Source}] Falló la extracción tras {Elapsed} ms",
        extractor.SourceName, stopwatch.ElapsedMilliseconds);

    return ExtractionResult.Fail(
        extractor.SourceName, extractor.SourceType, ex.Message, stopwatch.ElapsedMilliseconds);
}
```

6.2 Extracción de CSV con validación tolerante

SistemaOpiniones.Etl/Extractors/CsvExtractor.cs

```
ReadingExceptionOccurred = args =>
{
    var row = args.Exception.Context?.Parser?.Row;

    if (args.Exception is TypeConverterException)
    {
        context.RegisterRejected();
        _logger.LogWarning("Fila {Row} descartada en {Source}: {Message}", row, SourceName,
            args.Exception.Message);
    }
    else
    {
        _logger.LogWarning("Fila {Row} con formato irregular en {Source}: {Message}", row,
            SourceName, args.Exception.Message);
    }

    return false; // no aborta la lectura del archivo
}
```


Se distingue el fallo de conversión —que sí descarta el registro— del cambio en el número de columnas, que CsvHelper reporta pero igual entrega la fila. Sin esa distinción los totales del reporte no cuadrarían con las filas del archivo.

6.3 Extracción de base de datos en streaming

SistemaOpiniones.Etl/Extractors/DatabaseExtractor.cs

```
await using var connection = new SqlConnection(connectionString);
await connection.OpenAsync(cancellationToken);

await using var command = new SqlCommand(_options.Query, connection)
{
    CommandType = CommandType.Text,
    CommandTimeout = _options.CommandTimeoutSeconds
};

await using var reader = await command.ExecuteReaderAsync(cancellationToken);
var ordinals = MapOrdinals(reader); // se resuelven una sola vez

while (await reader.ReadAsync(cancellationToken))
{
    yield return new RawOpinion { /* ... */ };
}
```

6.4 Consumo de API con reintentos

SistemaOpiniones.Etl/Extractors/ApiExtractor.cs

```
if (!response.IsSuccessStatusCode)
{
    if (!IsTransient(response.StatusCode) || attempt > _options.RetryCount)
        throw new HttpRequestException($"La API respondió {(int)response.StatusCode} {response.ReasonPhrase} en '{url}'");

    await DelayBeforeRetryAsync(attempt, $"HTTP {(int)response.StatusCode}", cancellationToken);
    continue;
}
```

6.5 Registro de fuentes y manejo de credenciales

SistemaOpiniones.Etl/Program.cs

```
builder.Services.AddTransient<IExtractor, CsvExtractor>();
builder.Services.AddTransient<IExtractor, DatabaseExtractor>();
builder.Services.AddTransient<IExtractor, ApiExtractor>();

builder.Services.AddHttpClient(ApiExtractor.HttpClientName, (serviceProvider, client) =>
{
    var api = serviceProvider.GetRequiredService<IOptions<EtlOptions>>().Value.Sources.Api;

    client.BaseAddress = new Uri(api.BaseUrl, UriKind.Absolute);
    client.Timeout = TimeSpan.FromSeconds(api.TimeoutSeconds);

    // La llave nunca se registra en logs ni se guarda en appsettings.json.
    if (!string.IsNullOrEmpty(api.ApiKey))
```

```
        client.DefaultRequestHeaders.Add(api.ApiKeyHeaderName, api.ApiKey);  
    })  
    .SetHandlerLifetime(TimeSpan.FromMinutes(5));
```

7. Evidencia de ejecución

Resumen registrado por el propio servicio al final de una corrida real:

```
[INF] Lote 20260806-032555-540cdb: iniciando extracción de 3 fuente(s) en paralelo
[INF] ————— Resumen del lote 20260806-032555-540cdb —————
[INF] EncuestasInternas      CSV      OK      39 extraídos    1 descartados    138 ms
[ERR] ResenasWeb           Database ERROR    No se encontró el servidor o no estaba
accesible
[INF] ComentariosRedesSociales ApiRest OK      500 extraídos    0 descartados    900 ms
[INF] TOTAL: 539 registros, 1 descartados, 1 fuente(s) con error en 15182 ms
      (suma secuencial 16176 ms; el paralelismo ahorró 994 ms)
```

Registro aterrizado en staging, tal como queda en el archivo NDJSON:

```
{"BatchId": "20260806-032555-540cdb", "SourceName": "EncuestasInternas", "SourceType": "CSV",
  "ExternalId": "1", "ClienteRef": "7", "ProductoRef": "16", "FuenteRef": "Encuesta",
  "Fecha": "2025-01-10", "Comentario": "El proceso de compra fue muy sencillo, repetiría.",
  "Puntaje": "5", "Clasificacion": "Positiva", "ExtractedAtUtc": "2026-08-
06T03:25:55.545Z", "RawPayload": null}
```

El archivo de encuestas tiene 40 filas de datos; se extrajeron 39 y se descartó 1, la única con un identificador de cliente no numérico. Los totales cuadran con el contenido del archivo, que es la comprobación de que el conteo de descartes es correcto.

La fuente de base de datos aparece con error porque el equipo donde se ejecutó la prueba no tiene SQL Server instalado. El resultado es, precisamente, la evidencia del **aislamiento de fallos**: la caída de una fuente no impide que las otras dos se extraigan con normalidad, y el resumen señala exactamente cuál falló y por qué.

8. Estado de la entrega

Requisito de la actividad	Estado
Worker Service en .NET 8	Implementado — <code>SistemaOpiniones.Etl</code>
Extracción de CSV con CsvHelper	Implementado y ejecutado
Extracción desde base de datos relacional	Implementado; pendiente de servidor SQL para ejecutarlo
Extracción desde API REST con <code>HttpClient</code>	Implementado y ejecutado
Datos en archivos temporales o tablas staging	Ambos: NDJSON y <code>dbo.Stg_Opinion</code>
Logs de eventos y errores con <code>ILogger</code>	<code>ILogger</code> + Serilog (consola y archivo diario)
Interfaz <code>IExtractor</code> y clases derivadas	Implementado
Validación de rendimiento con <code>Stopwatch</code>	Implementado, por fuente y total

Configuraciones centralizadas en archivos JSON	<code>appsettings.json</code> + User Secrets
Diagrama de arquitectura	Sección 2
Diagrama de flujo del proceso ETL	Sección 3
Justificación de las decisiones técnicas	Sección 4
Validación de los atributos de calidad	Sección 5
Evidencia del código de extracción	Secciones 6 y 7

8.1 Pendiente

La fuente de base de datos no se ha podido **ejecutar** porque el equipo de pruebas no tiene SQL Server instalado. El código, el script de creación de la base de origen (`SistemaOpiniones.Etl/Sql/02_Origen_ResenasWeb.sql`) y su configuración están completos: al levantar el servidor y ajustar la cadena de conexión, la fuente entra en funcionamiento sin cambios de código.